

GNR638

Assignment 2

Mohammed Nasir (24M2158)

Pranav Bhagwat (24M2165)

Jatin Kumar (24M2162)

Convolutional Neural Network (CNN) backbone analysis

Three CNN architectures, all pre-trained on ImageNet, were selected for this assignment. EfficientNet-B0, DenseNet121, and ResNet50. The models were loaded using the timm library. All experiments were executed on a CUDA-enabled GPU with a fixed random seed (42) to ensure reproducibility.

The experiments were conducted using the Aerial Images Dataset (AID), consisting of 6,993 images distributed across 30 distinct classes.

The computational complexity and parameter count for each backbone are as follows:

Model	Total Parameters	MACs	FLOPs
EfficientNet-B0	4,045,978	384,637,152	769,274,304
DenseNet121	6,984,606	2,833,395,200	5,666,790,400
ResNet50	23,569,502	4,131,756,032	8,263,512,064

The experiments adhered strictly to the computational constraints. Full-data training was capped at 30 epochs. As we used GPU, our training time was well within limit.

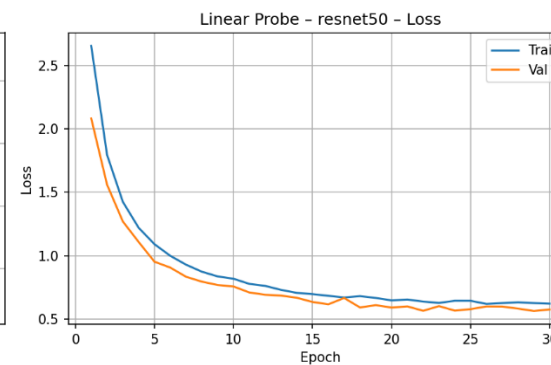
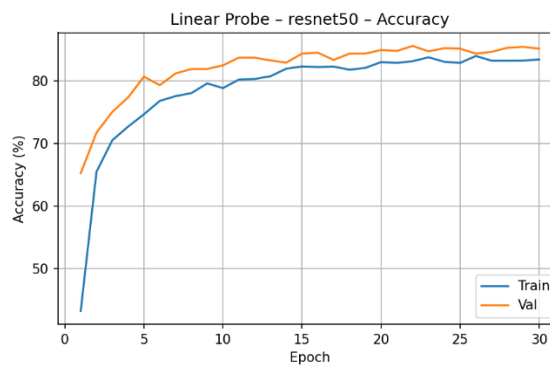
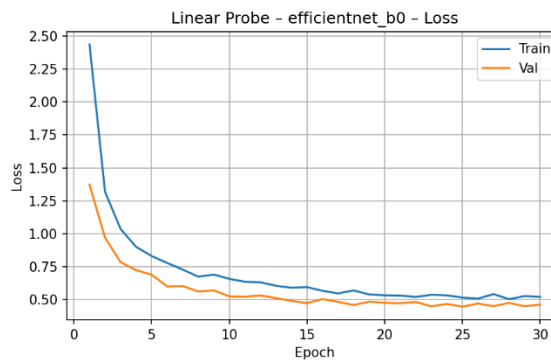
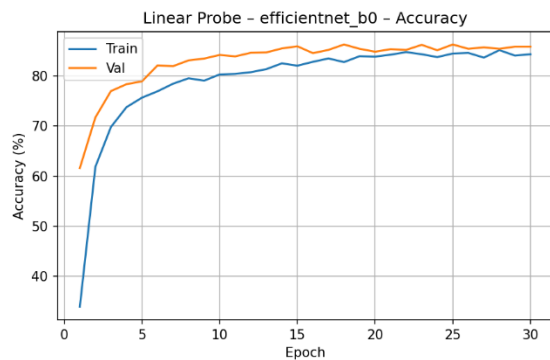
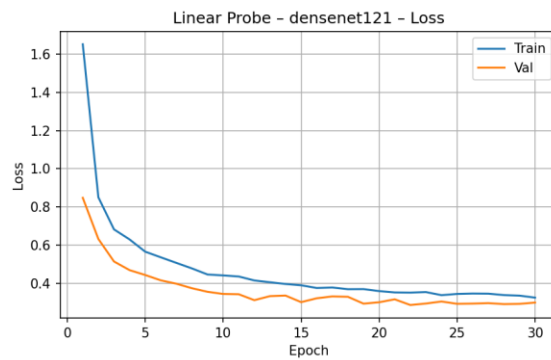
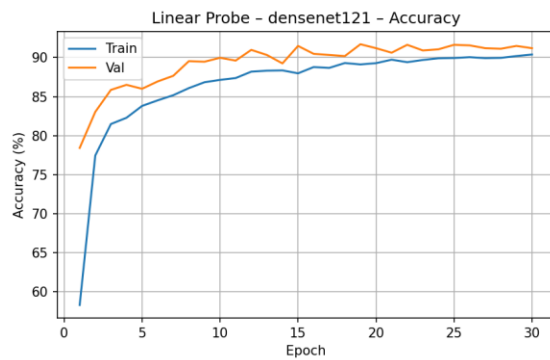
Scenario 1: Linear Probe Transfer

In this scenario, the transferability of pre-trained representations was evaluated by freezing all backbone parameters and training only a newly initialized linear classification head for 30 epochs.

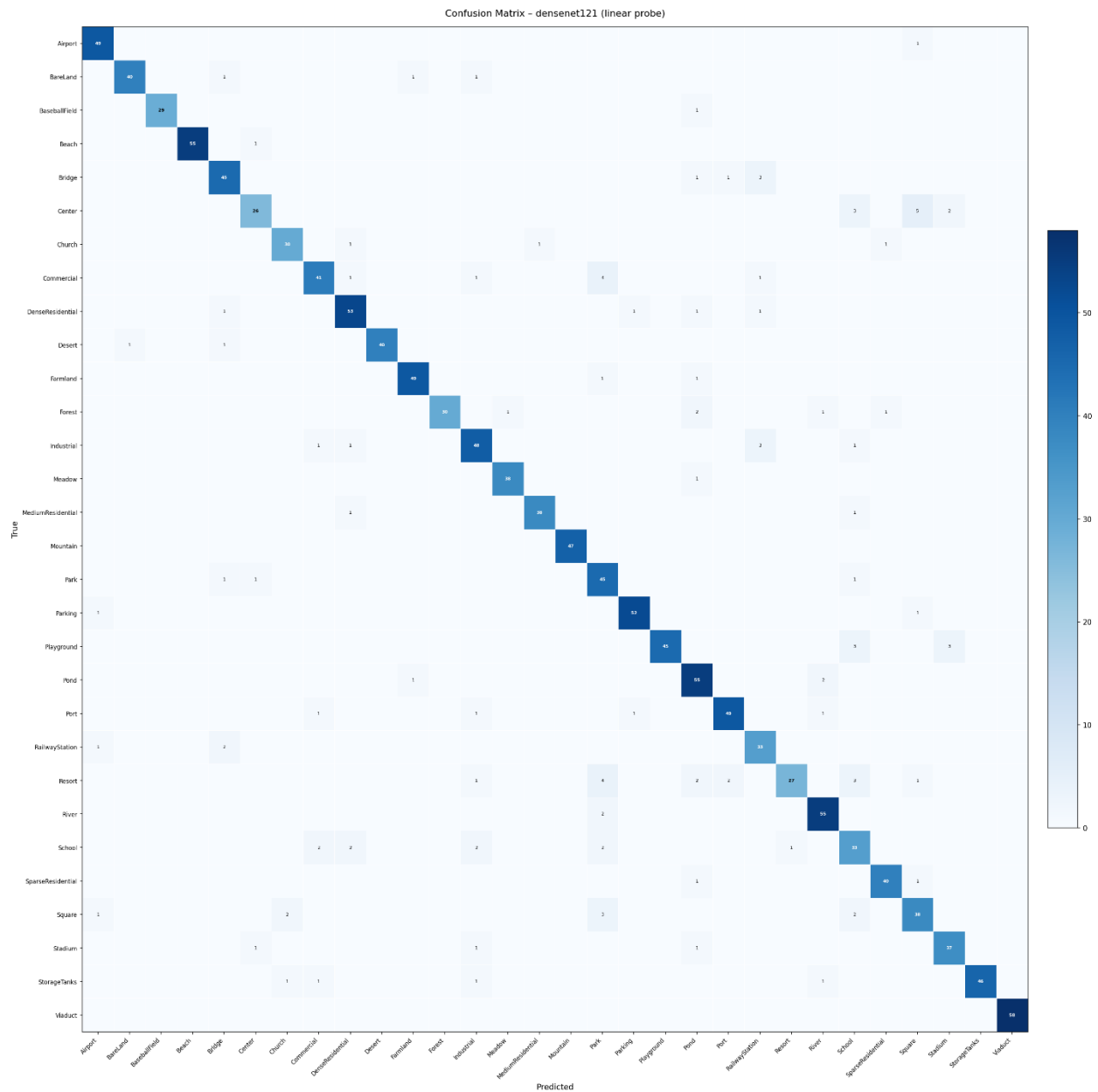
Performance Summary

Model	Final Train Accuracy	Final Validation Accuracy	Peak Validation Accuracy (Epoch)
EfficientNet-B0	84.34%	85.86%	86.29% (25)
DenseNet121	90.39%	91.20%	91.70% (19)
ResNet50	83.41%	85.14%	85.57% (22)

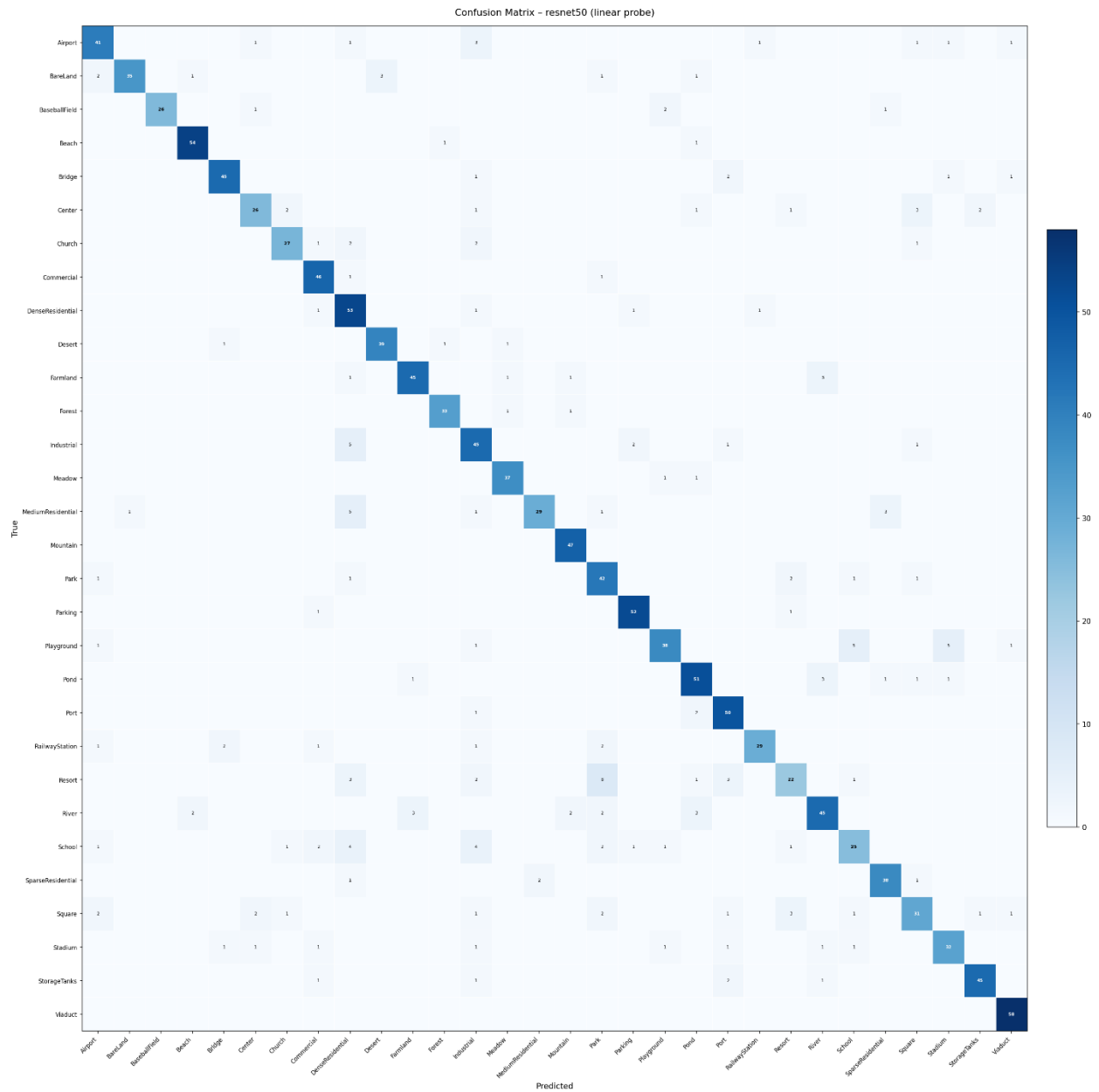
Accuracy and Loss Curves



Confusion matrix

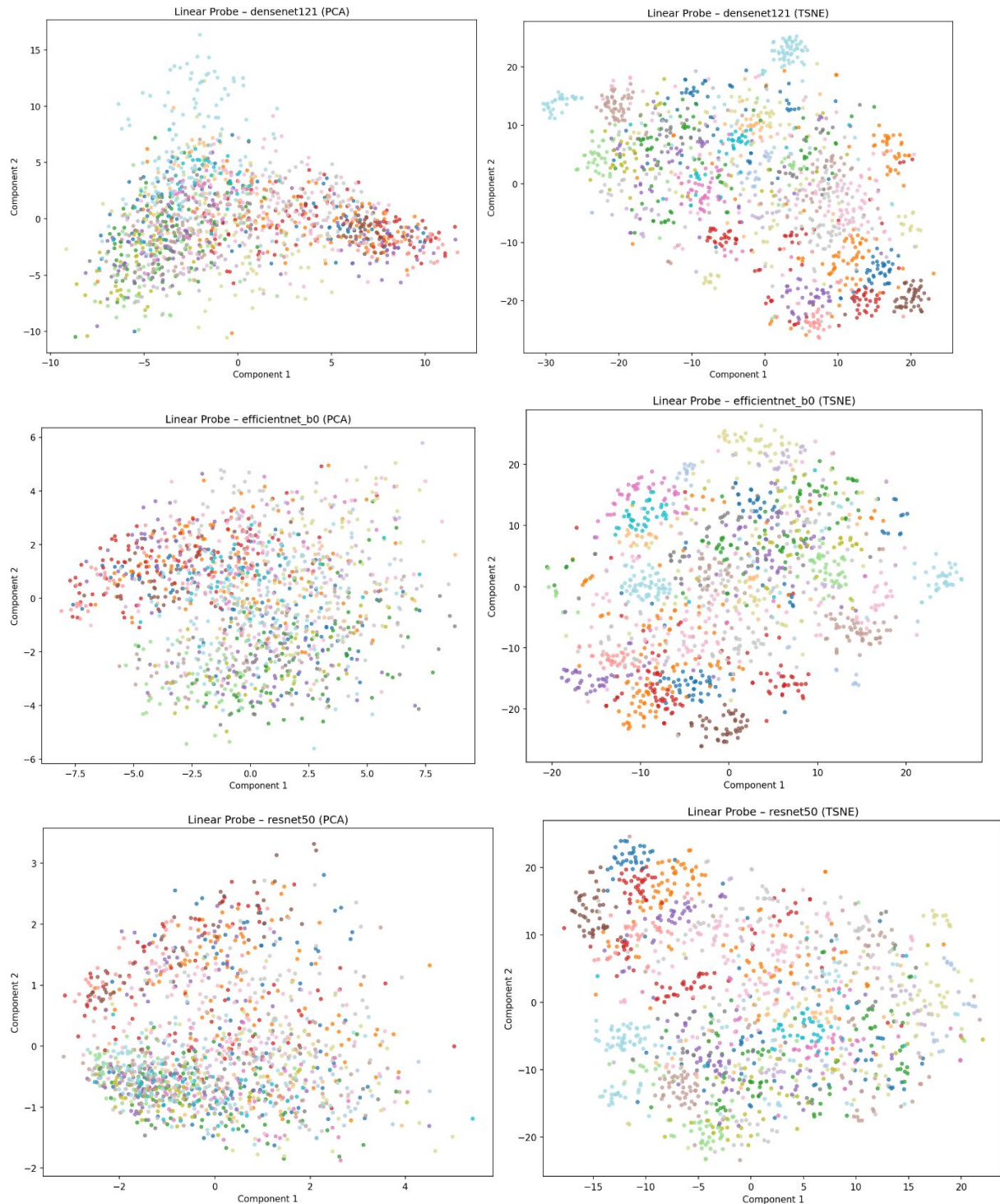


Confusion matrix for the 'Type' variable. The matrix shows counts for 30 predicted vs. actual categories. The diagonal is dark blue, indicating high accuracy. A color bar on the right shows the count scale from 0 to 50.



These figures depict Confusion matrix for densenet (top), efficientnet (middle) and resnet (bottom). All have dark blue diagonals, indicating good accuracy (which is also evident by accuracy table, Although DenseNet has better performance when measured)

PCA and tSNE Plot



Analysis of Feature Separability and Transferability:

When used strictly as frozen feature extractors, all three models demonstrated strong baseline transferability to the aerial imagery domain without any backbone

adaptation. DenseNet121 yielded the highest representation quality, achieving a peak validation accuracy of 91.70%, notably outperforming both EfficientNet-B0 and ResNet50 by approximately 5-6%.

This suggests that the dense connectivity pattern in DenseNet121 preserves more generic, highly transferable semantic features from ImageNet that map effectively to top-down aerial viewpoints.

Scenario 2: Fine-Tuning Strategies

This scenario investigates layer-wise adaptation by comparing four distinct fine-tuning strategies across the three selected architectures: Linear Probe (frozen backbone), Last Block fine-tuning, Selective 20% fine-tuning (unfreezing the top ~20% of parameters), and Full Fine-tuning (unfreezing 100% of parameters).

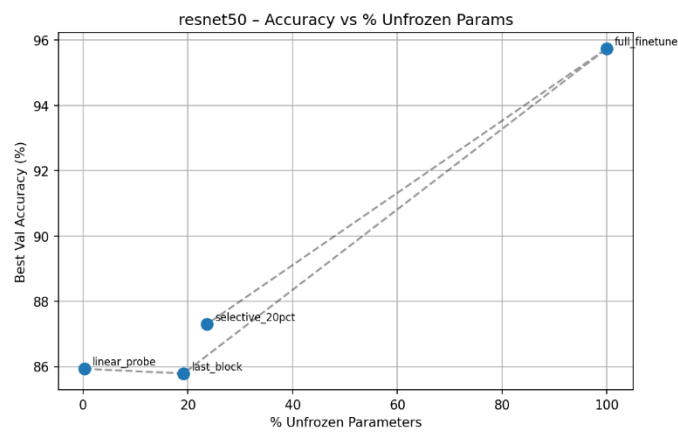
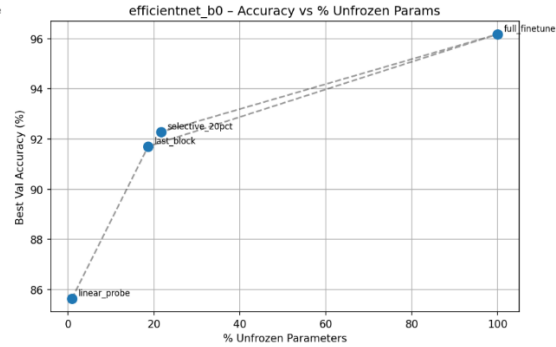
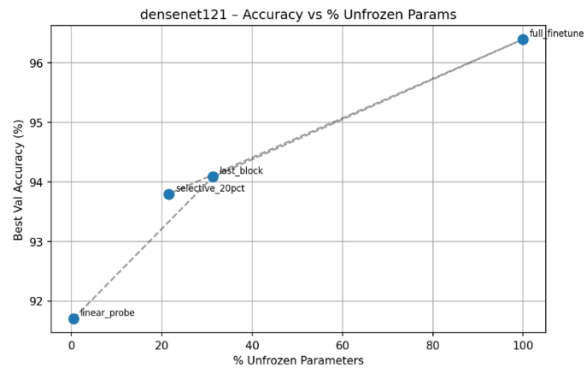
Tuning Strategy for the Selective 20% Unfreeze: The deepest ~20% of backbone parameters were selected because deeper layers encode task-specific, high-level semantics and should yield the highest accuracy gain per additional trainable parameter.

Performance Summary:

The table below summarizes the final validation accuracy achieved at Epoch 30 for each strategy.

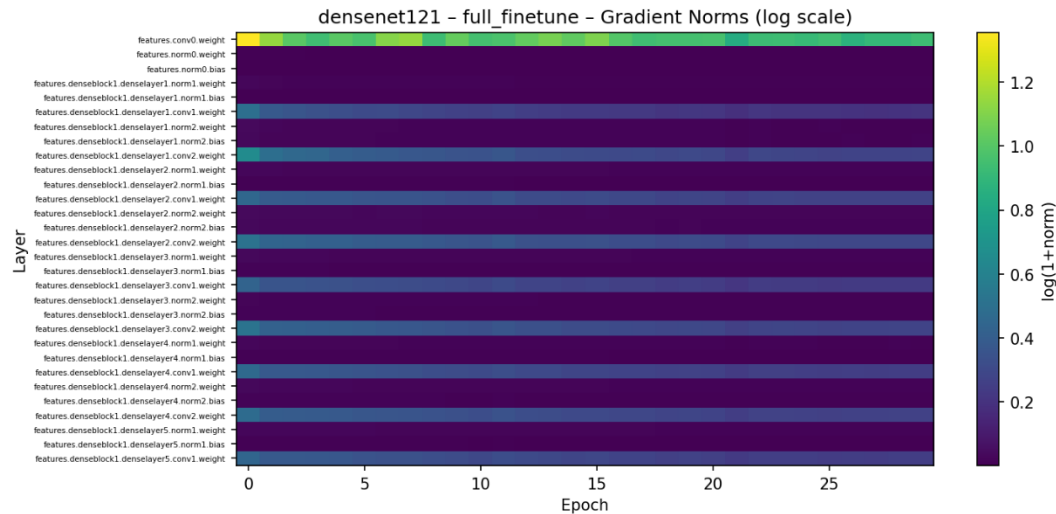
Model	Linear Probe	Selective 20%	Last Block	Full Fine-Tuning
EfficientNet-B0	85.64%	91.56%	91.05%	95.60%
DenseNet121	90.98%	93.43%	93.43%	96.10%
ResNet50	85.21%	87.16%	84.20%	95.45%

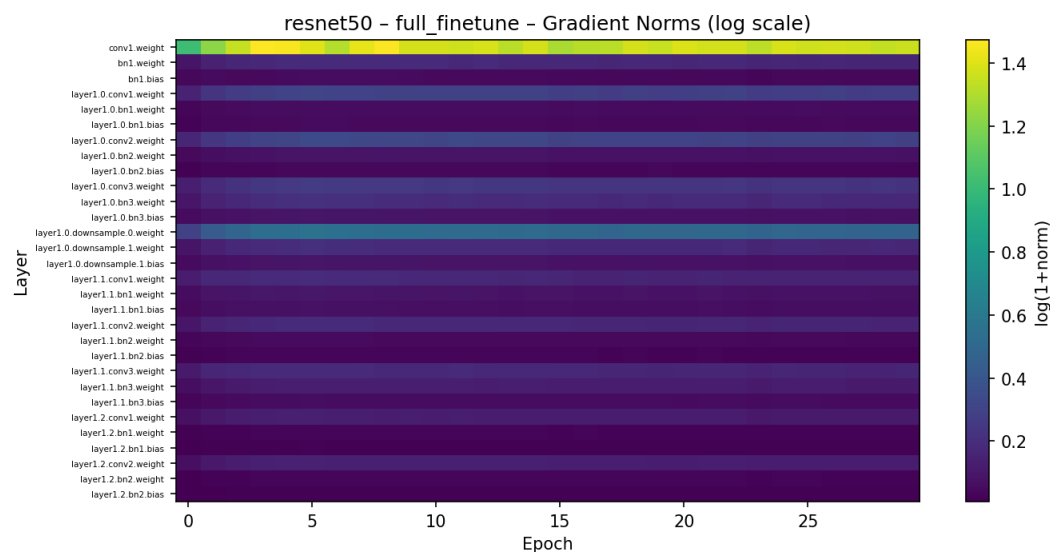
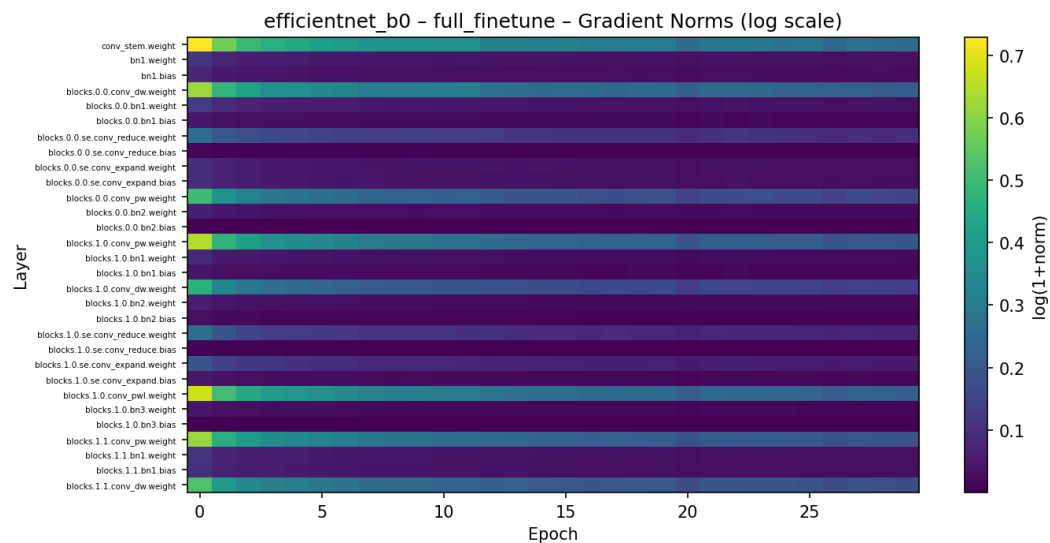
Strategy Comparison for Accuracy



These figures depict strategy comparison plot for densenet (top left), efficientnet (top right) and resnet (bottom left). Full fine tuning is best in all cases, and “selected 20%” strategy is second best in efficientnet and resnet.

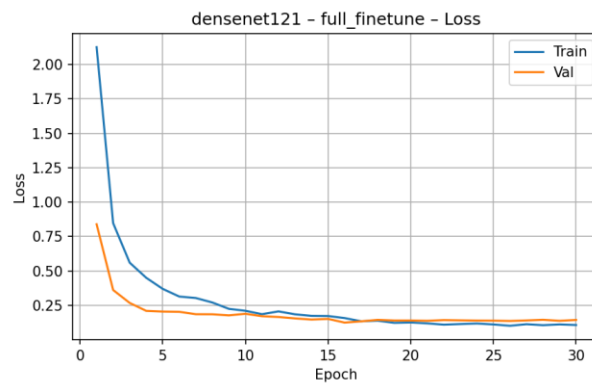
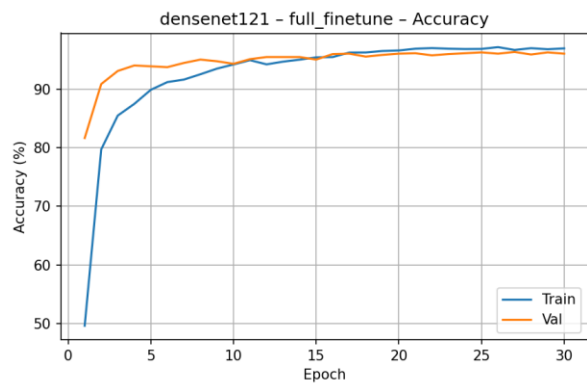
Grad Norms

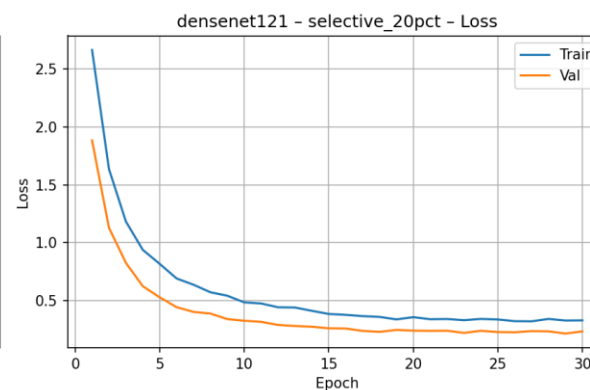
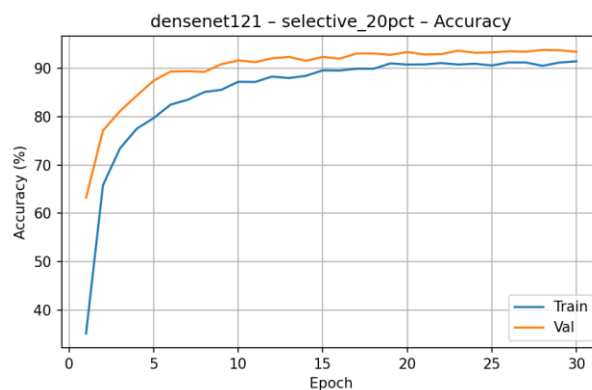
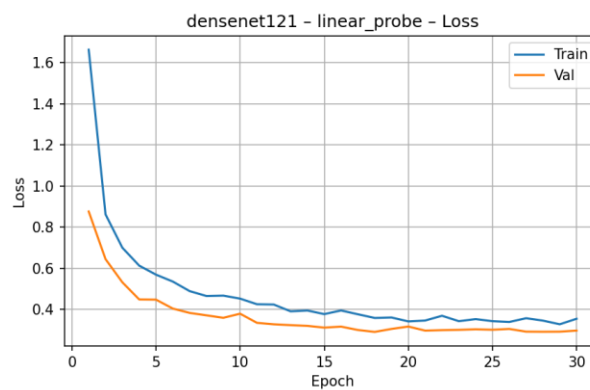
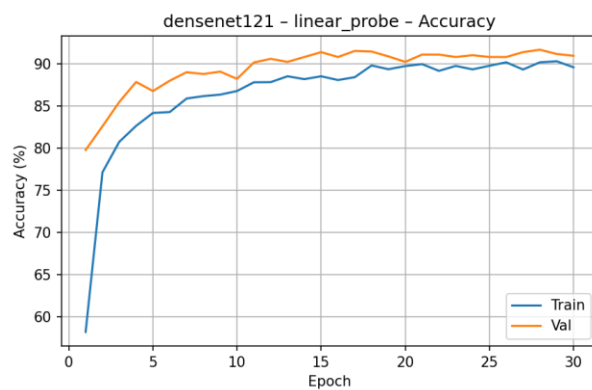
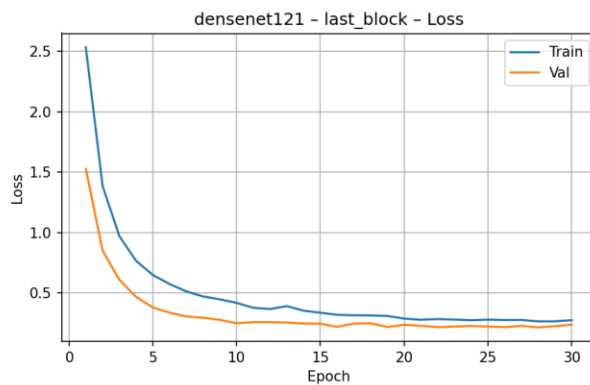
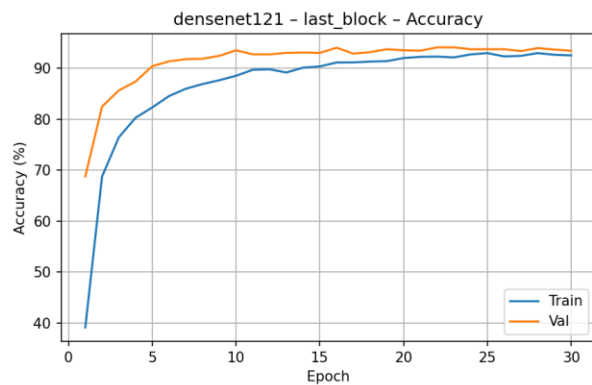




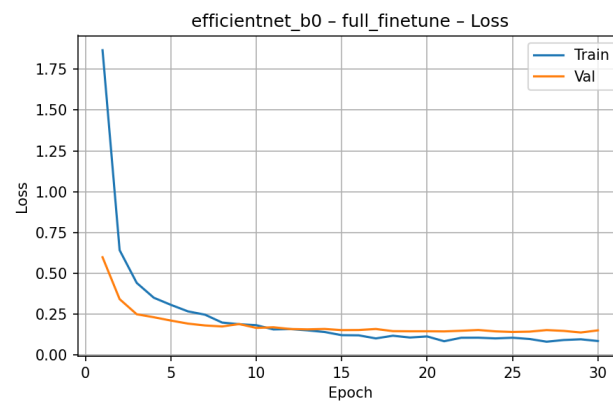
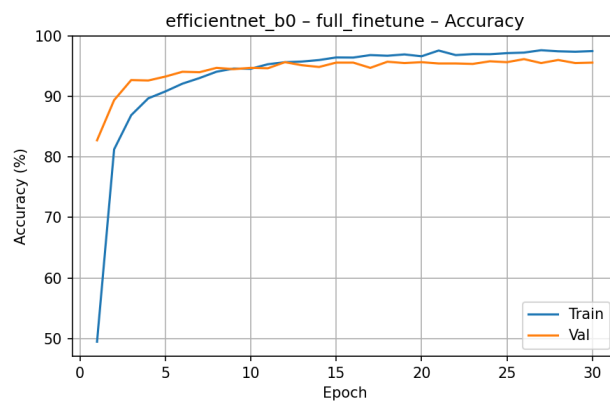
Accuracy and Loss curves

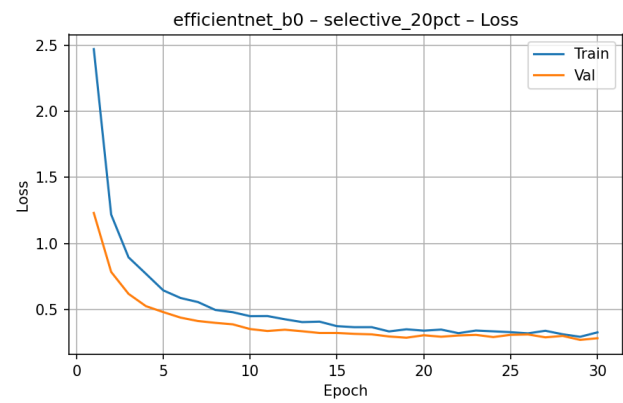
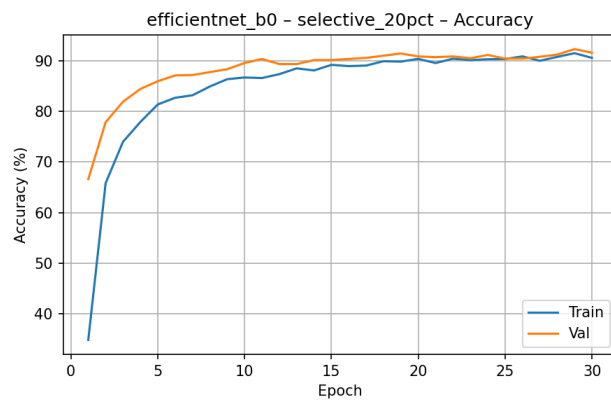
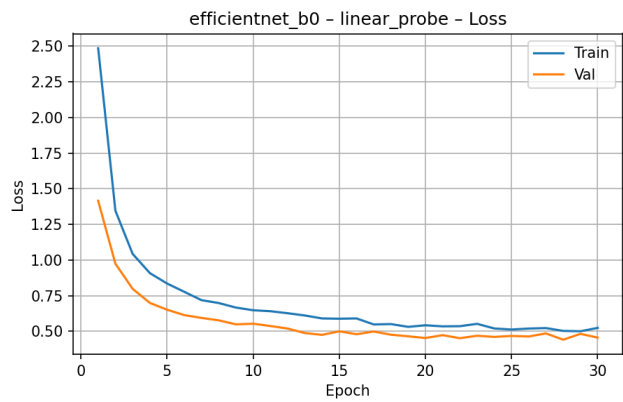
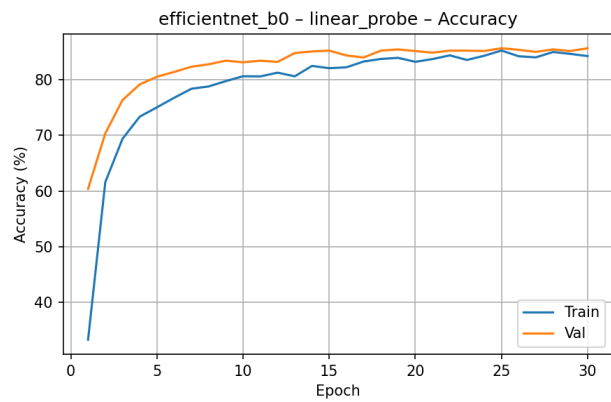
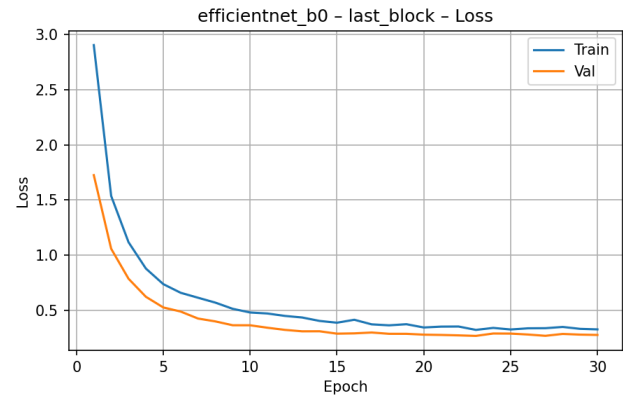
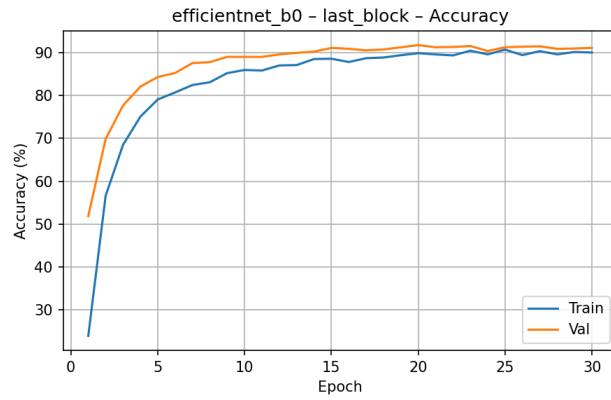
Densenet121



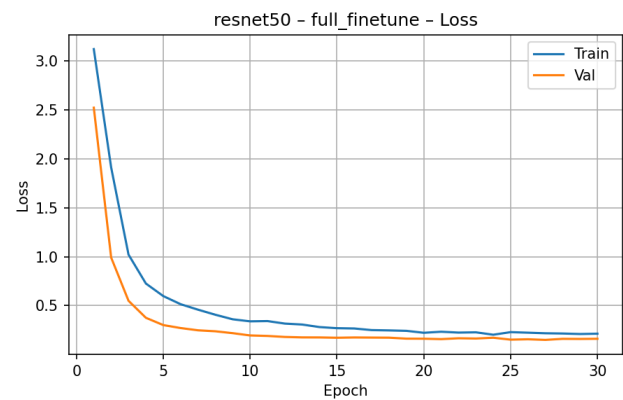
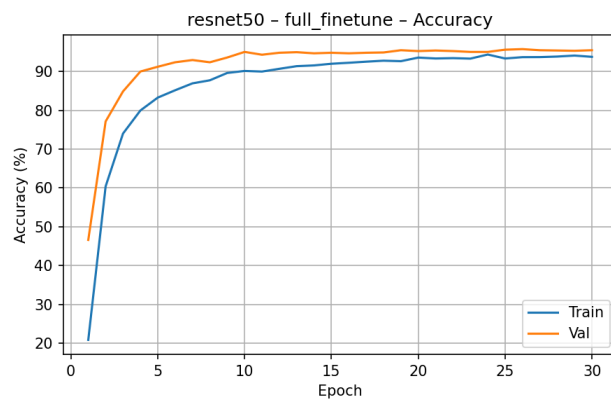


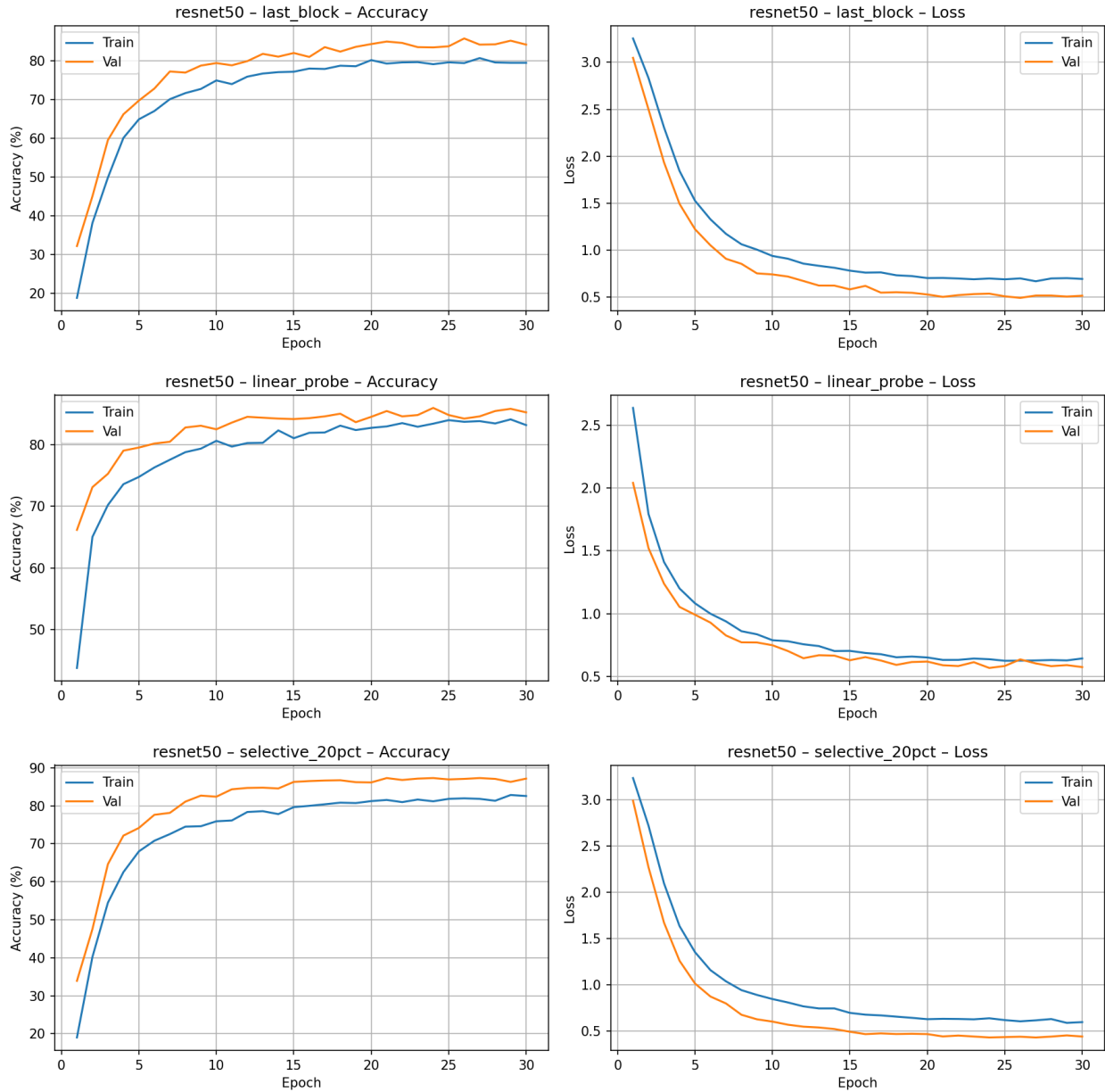
Efficientnet_b0





Resnet50





Analysis of Layer-Wise Adaptation and Convergence:

- Full Fine-Tuning shows best performance across all three architectures showing that adapting the entire feature hierarchy is highly beneficial. DenseNet121 maintained its lead overall, reaching a peak validation accuracy of 96.39%. while other two architectures saw high jump, from ~85% (Linear Probe) to ~95% (Full Fine-Tune).
- For EfficientNet-B0 and DenseNet121, unfreezing just the deeper layers (Last Block) or a calculated 20% slice (Selective 20%) closed the majority of the gap

between the Linear Probe and Full Fine-Tuning. This indicates that while the generic early-layer ImageNet features are highly transferable, representation quality is significantly improved by adapting the high-level semantic features in the deeper layers to the specific geometries of dataset.

- ResNet50 had an anomaly as fine-tuning the Last Block (84.20%) resulted in slightly *worse* performance than the simple Linear Probe (85.21%). However, the Selective 20% strategy yielded a stable improvement (87.16%). This suggests that ResNet50's Last Block does not capture a complete semantic layer block for meaningful gradient updates, causing the learned representations to degrade.
- The acc_loss curves reveal that while Full Fine-Tuning achieves the highest accuracy, it also shows signs of early over-fitting, with validation losses flattening early while training accuracy continued to climb. The partial unfreezing strategies demonstrated more stable, but slower convergence.
- While Full Fine-Tuning maximizes validation accuracy on the clean target dataset, it reduces robustness to out-of-distribution shifts. By updating 100% of the weights, the model overwrites generalized feature space learned from the massive ImageNet dataset, overfitting heavily to the narrow distribution of the given dataset.

Scenario 3: Few-Shot Learning Analysis

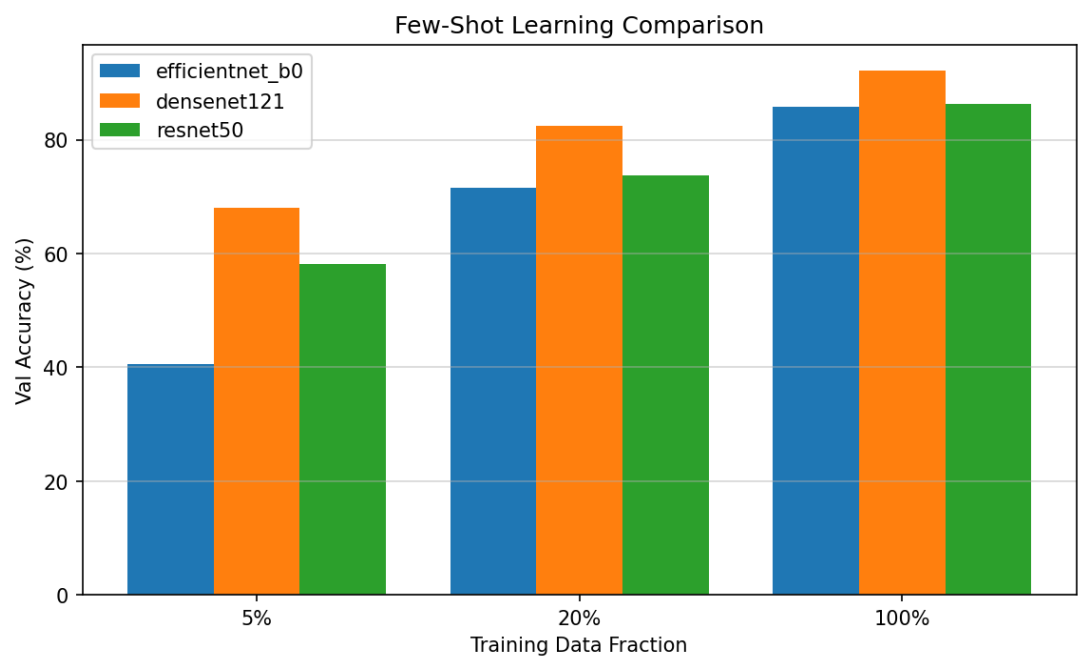
This scenario evaluates the data efficiency and generalization capability of each backbone under limited supervision regimes. The models were trained using 100%, 20%, and 5% of the total available training data in a linear probe setting. Also, for 5% and 20% data setting, only 20 epochs were trained, to stay within constraint.

Performance Summary

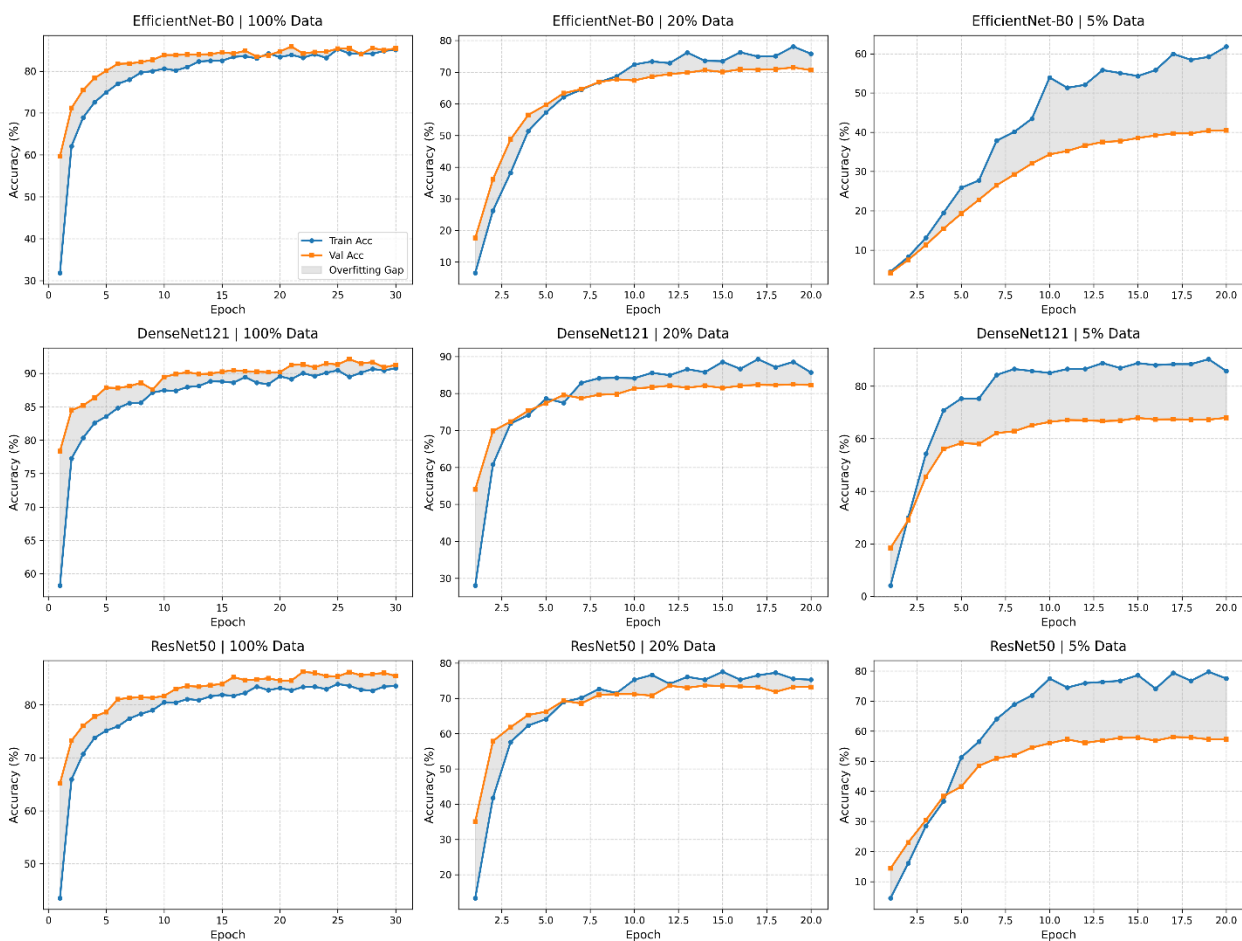
The following table presents the peak validation accuracy achieved for each model under varying training data constraints, alongside the calculated relative performance drop between the full dataset and the 5% few-shot setting.

Model	100% Data	20% Data	5% Data	Relative Performance Drop (100%→5%)
EfficientNet-B0	85.86%	71.50%	40.48%	52.86%
DenseNet121	92.14%	82.47%	67.97%	26.23%
ResNet50	86.22%	73.67%	58.08%	32.64%

Performance Comparison visualised



Scenario 4.3: Few-Shot Training vs Validation Gap



Analysis of Data Efficiency and Overfitting:

- DenseNet121 demonstrated much better data efficiency compared to the other architectures. When trained on merely 5% of the data, DenseNet121 maintained a validation accuracy of 67.97%, limiting its relative performance drop to just 26.23%. This indicates that the dense block structure, where each layer has direct access to the gradients from the loss function and the original input signal, creates highly expressive feature representations.
- Model Capacity and The Training-Validation Gap: The impact of limited data on overfitting is clearly observable in the training-validation gaps at the 5% regime. All three architectures exhibit a gap of $\sim 21\%$. DenseNet121 did so while maintaining a significantly higher absolute accuracy threshold.

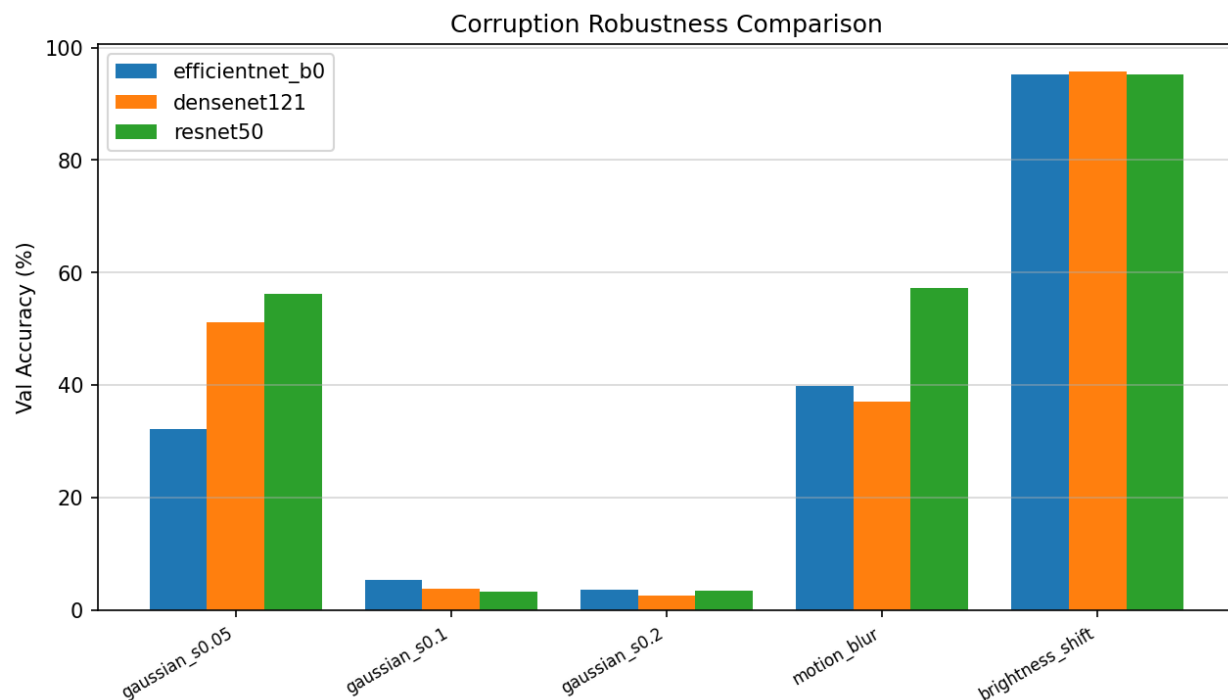
Scenario 4: Corruption Robustness Evaluation

In this scenario, model robustness was tested under controlled distribution shifts applied strictly during evaluation time. The fully fine-tuned models were exposed to pixel-level Gaussian noise ($\sigma = 0.05, 0.1, 0.2$), motion blur, and brightness shifts. Performance is evaluated using Validation Accuracy, Corruption Error (CE), and Relative Robustness (RR).

Model	Clean Acc	($\sigma=0.05$)	($\sigma=0.1$)	($\sigma=0.2$)	Motion Blur	Brightness Shift
EfficientNet-B0	96.18%	Acc:	Acc:	Acc:	Acc:	Acc:
		32.11%	5.27%	3.61%	39.83%	95.24%
		CE:	CE:	CE:	CE:	CE:
		0.6789	0.9473	0.9639	0.6017	0.0476
DenseNet121	96.39%	RR:	RR:	RR:	RR:	RR:
		0.3338	0.0548	0.0375	0.4141	0.9902
		Acc:	Acc:	Acc:	Acc:	Acc:
		51.15%	3.75%	2.60%	37.01%	95.82%
		CE:	CE:	CE:	CE:	CE:
		0.4885	0.9625	0.9740	0.6299	0.0418
		RR:	RR:	RR:	RR:	RR:
		0.5307	0.0389	0.0269	0.3840	0.9940

ResNet50	95.74%	Acc: 56.20% CE: 0.4380 RR: 0.5870	Acc: 3.25% CE: 0.9675 RR: 0.0339	Acc: 3.32% CE: 0.9668 RR: 0.0347	Acc: 57.22% CE: 0.4278 RR: 0.5976	Acc: 95.17% CE: 0.0483 RR: 0.9940
-----------------	--------	--	---	---	--	--

Performance Comparison visualised



Analysis of Corruption Sensitivity:

- All models exhibited extreme vulnerability to high-level Gaussian noise (0.1, 0.2), with accuracy dropping to near-random guessing (2-5%). This highlights a shared architectural sensitivity to high-frequency pixel perturbations.
- However, structural differences were seen clearly under low-level noise setting (0.05) and motion blur. ResNet50 emerged as the most robust architecture under these specific shifts. It retained Relative Robustness of 0.5870 under sigma=0.05 noise, and Relative Robustness 0.5976 under motion blur, and greatly outperformed EfficientNet-B0 and DenseNet121 in both these settings. This suggests that the standard residual connections in ResNet50, which allow identity mapping, provide a more robust defense against spatial blurring than

the feature-concatenation method used in DenseNets or the depth-wise convolutions in EfficientNets.

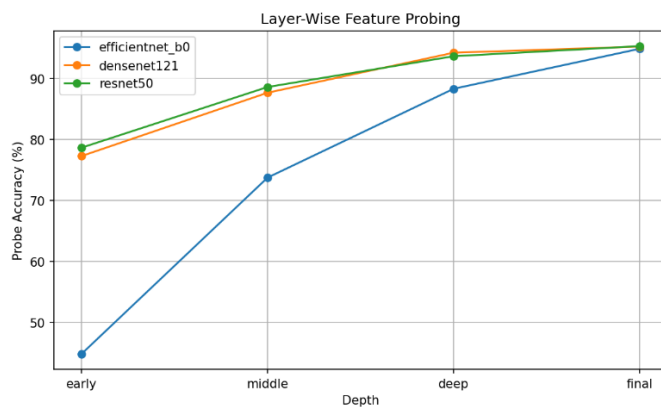
- All models proved highly robust to brightness shifts, indicating that simple luminance scaling does not significantly disrupt learned spatial features.

Scenario 5: Layer-Wise Feature Probing

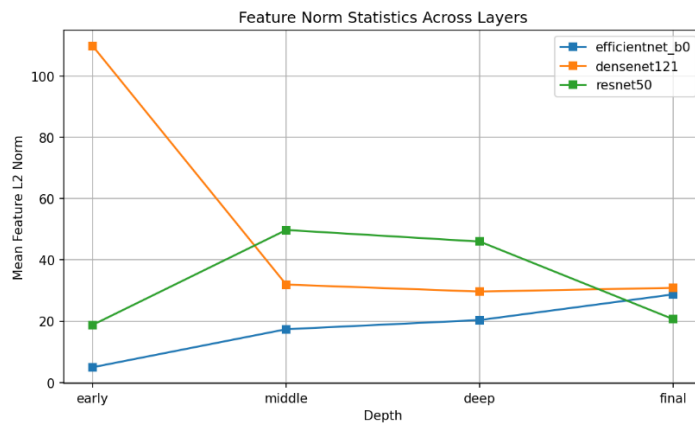
To examine semantic abstraction, intermediate representations were extracted from early, middle, deep, and final layers of the fully fine-tuned models. Separate linear classifiers were trained on these extracted features.

Performance Summary

Model	Early Layer Acc.	Middle Layer Acc.	Deep Layer Acc.	Final Layer Acc.
EfficientNet-B0	44.81%	73.74%	88.31%	94.88%
DenseNet121	77.27%	87.66%	94.23%	95.24%
ResNet50	78.64%	88.60%	93.65%	95.31%



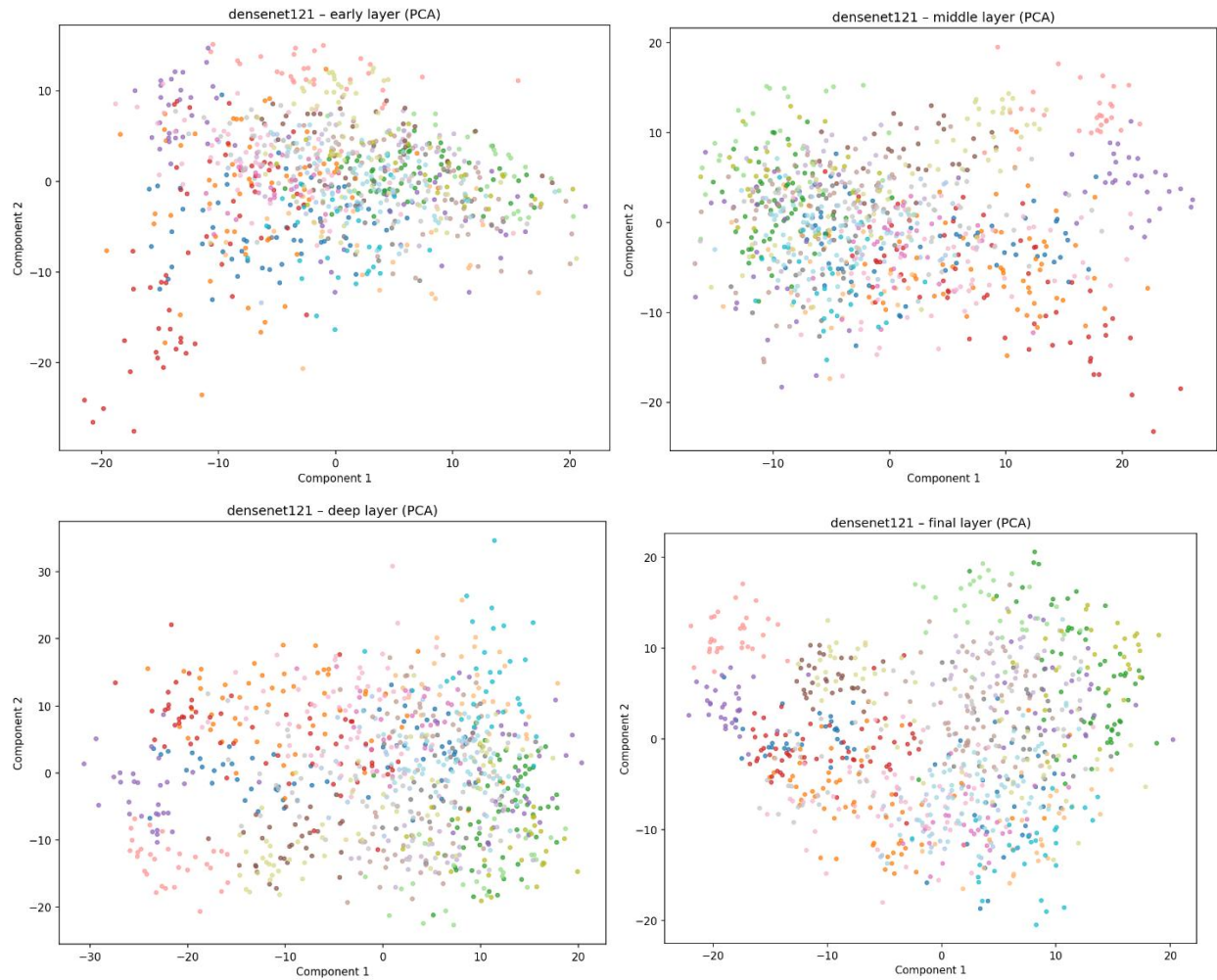
Above Figure depicts Probe accuracy vs Depth of all 3 architectures



Above Figure depicts Feature norms across Layers for all 3 architectures

PCA Plots

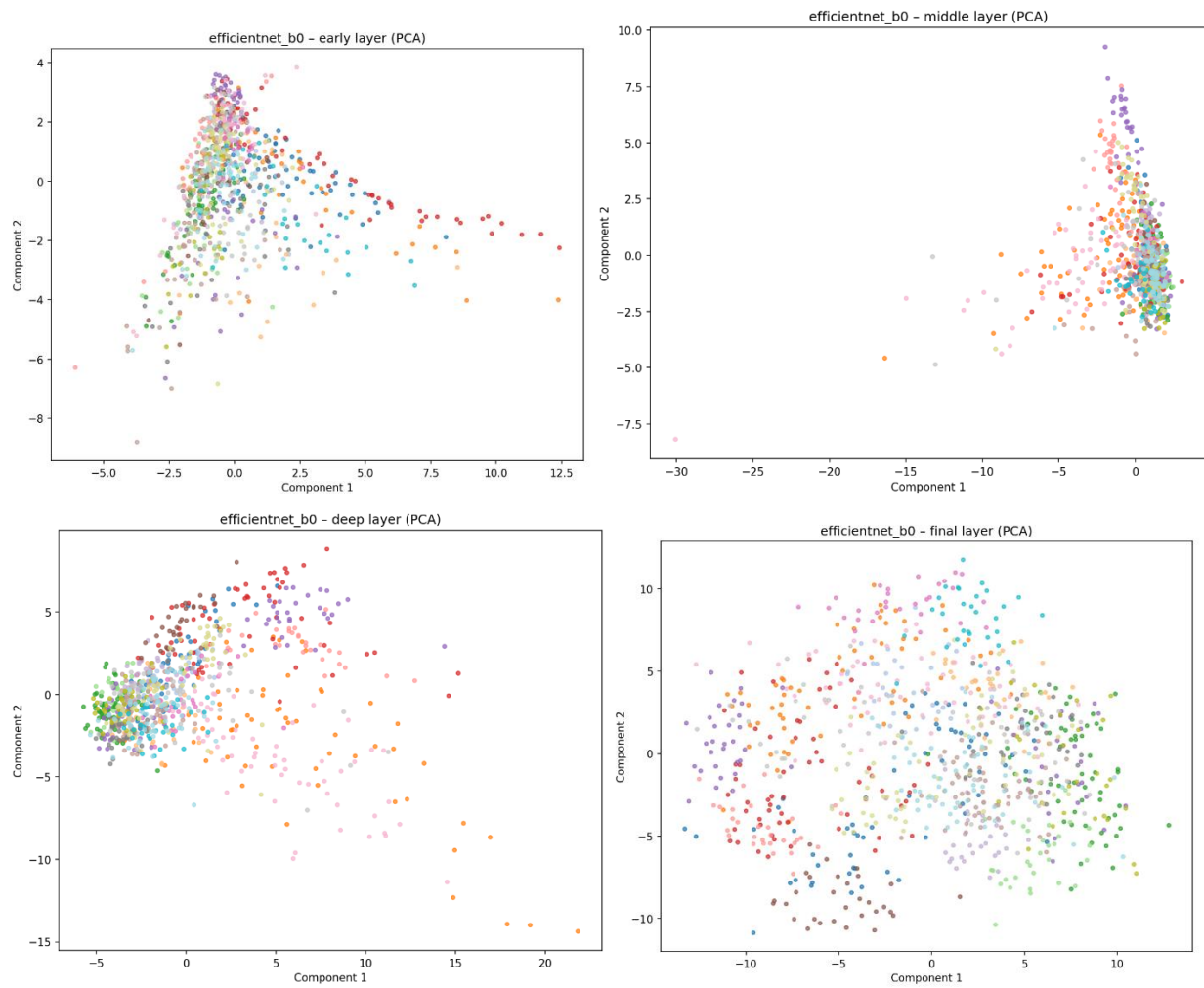
Densenet121



Above figures depict DenseNet Layerwise PCA for: early layer (top left), middle layer (top right), deep layer (bottom left), final layer (bottom right).

We see that in early layer, most classes are tangled.

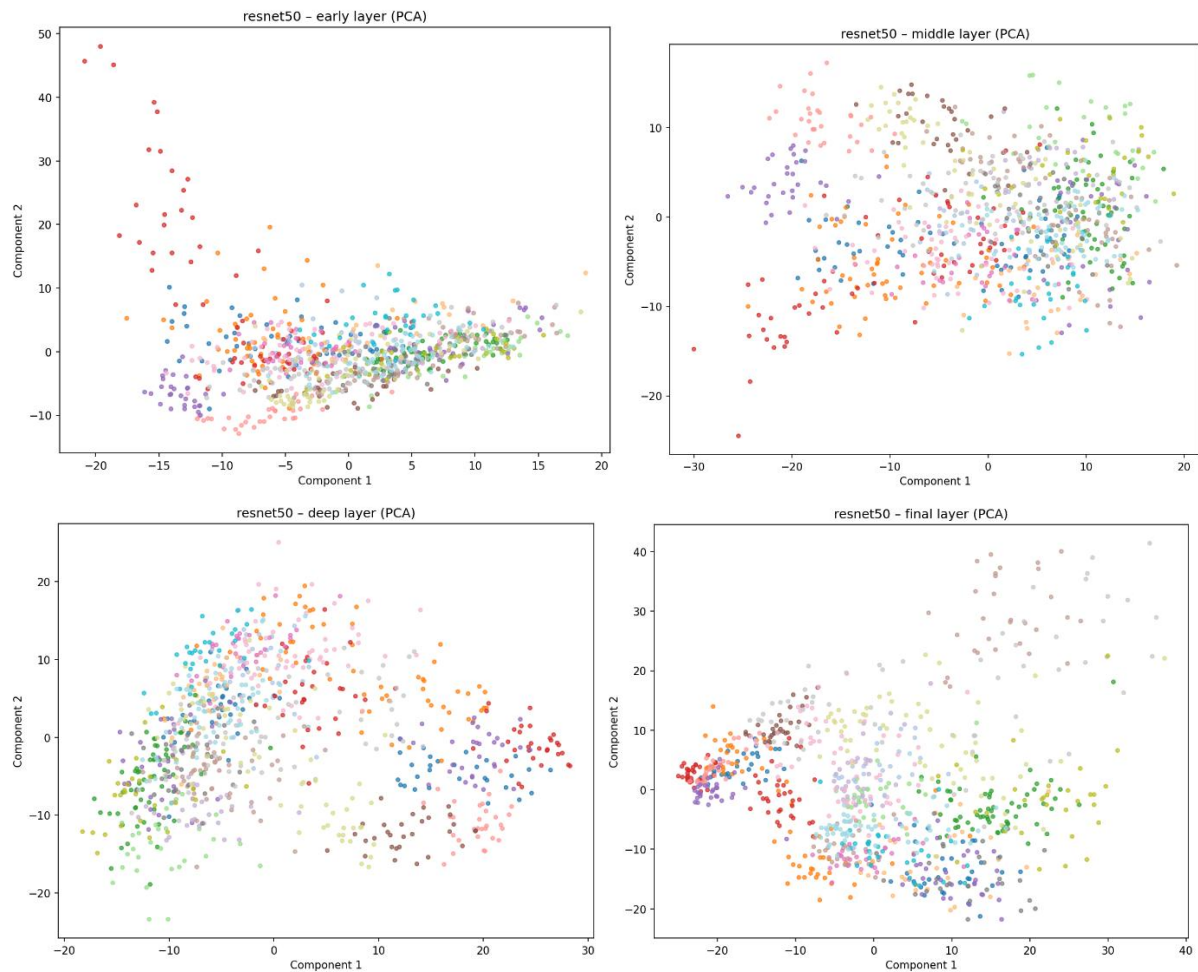
Efficientnet_b0



Above figures depict efficientNet Layerwise PCA for: early layer (top left), middle layer (top right), deep layer (bottom left), final layer (bottom right).

We see that only in final layer, most classes are untangled, Rest of cases they are not separated well.

Resnet50



Above figures depict resnet Layerwise PCA for: early layer (top left), middle layer (top right), deep layer (bottom left), final layer (bottom right).

We see that only in early layer, most classes are tangled, while in later layers classes start separating.

Analysis of Semantic Abstraction

- Classification performance increase monotonically with network depth across all models. Early layers primarily capture low-level features (edges, textures), resulting in poor separation of complex classes (as seen in the early PCA plots where clusters are highly entangled).
- DenseNet121 and ResNet50 achieve high accuracy (77-78%) much earlier in their network depth than EfficientNet-B0 (44.81%). This indicates that larger

models with higher parameter counts encode more discriminative semantics in their earlier feature maps.

- As we progress to the middle and deep layers, the PCA visualizations are less entangled, showing the hierarchy of feature abstraction transitioning from general patterns to more specific representation.

Unexpected Finding

An unexpected finding was ResNet50's severe instability during the Last Block fine-tuning. Unlike EfficientNet and DenseNet, which saw immediate improvements, tuning only the final layer block of ResNet50 resulted in worse performance than a simple linear probe. This suggests that the architectural boundaries of a block in ResNet50 are not made to select specific features so it causes worse gradient updates when analyzed block-wise.

Failure Case

All architectures uniformly failed to handle high-level Gaussian noise (0.1, 0.2). The models collapsed to near 2-5% accuracy. This failure occurred because standard CNNs are biased toward high-frequency spatial features (like sharp edges and textures) for classification. When these high-frequency signals are corrupted by severe pixel-level noise, the core features the models rely on are destroyed, rendering them entirely incapable of making meaningful predictions.

Efficiency vs. Performance Analysis for Scenario 2

When evaluating the tradeoff between computational cost (parameters tuned) and performance gains, the Selective 20% strategy proved to be the efficient approach across all models. For example, in EfficientNet-B0, tuning just 876,318 parameters (Selective 20%) achieved an accuracy of 91.56%. Tuning all 4,045,978 parameters (Full Fine-Tune) only yielded an additional 4% gain (95.60%), but required nearly 5x the trainable parameter overhead. DenseNet121 exhibited a similar trend, where tuning roughly 1.5M parameters (Selective 20%) achieved 93.43%, nearly matching the 96.10% accuracy of the fully fine-tuned 6.9M parameter model. This demonstrates that adapting just the highest-level semantic layers yields the optimal balance of efficiency and accuracy.

Key Architectural Insights

- DenseNet121 proved to be the best at learning from very little data (Scenario 3) because it constantly passes information from early layers directly to the final layers, ensuring it never loses track of basic shapes and textures. In contrast, EfficientNet-B0 is built to be as small and fast as possible (4M parameters), but we found that this makes it fragile, it struggled the most when training data was scarce or when images were corrupted.
- ResNet50 demonstrated a toughness against motion blur because of its skip connections. These connections act like a safety net, allowing the original image details to bypass certain layers, proving that ResNet's design is naturally better at handling blurry or distorted spatial information.

Summary questions & answers

1) Which architecture transfers best and why?

DenseNet121 transferred best (as seen in Scenario 1: Linear Probe Transfer). Its unique design constantly reuses features from earlier layers, meaning it doesn't lose track of basic, highly transferable textures and shapes when mapping to a new dataset.

2) Which model is most robust under distribution shift?

ResNet50 proved to be the most robust against blurring and minor noise (as seen in Scenario 4: Corruption Robustness). Its built-in skip connections act as a safety net, allowing the original spatial details of the image to bypass the layers that get confused by the distortion.

3) Which layers encode the most transferable semantics?

The **early and middle layers** encode the most universally transferable features. These layers learn generic patterns like edges and basic textures, which easily apply to new tasks before the deeper layers get too specialized.

4) When does fine-tuning degrade robustness?

By updating 100% of the model's weights to perfectly fit the clean aerial images, the model **forgets its diverse ImageNet background** and becomes fragile when exposed to out of distribution samples.

5) How does inductive bias manifest empirically?

A model's core design directly dictates its real-world behavior. For instance, EfficientNet's bias toward **saving computational space** makes it fast but incredibly **fragile when training data is scarce**, whereas DenseNet's bias toward reusing information makes it highly sample-efficient.